

# Connected Devices and Wearables Lab, IIT- Bangalore

**Memo:** Mid-Term Report

**Category:** PE Term-II 2024-2025

**Title:** TinyML Code Implementation on PSoC-6 BLE

**Author-Name:** Bijet Basak

**Author-Email:** Bijet.Basak@iiitb.ac.in

**Date:** 25<sup>th</sup> March 2025

## Completed Work

### Summary

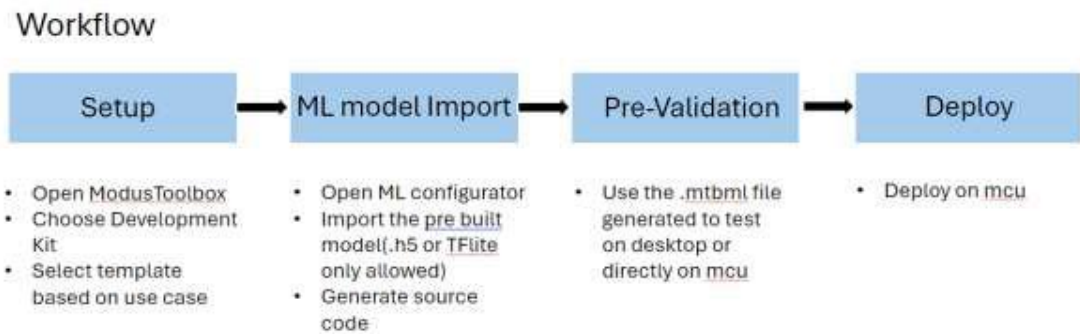
The project involves deploying a machine learning model on the **CY8PROTO-063-BLE** using **ModusToolbox**. The focus is on setting up the development environment, importing and validating the ML model, and deploying it onto the microcontroller unit (MCU). The key challenge encountered is the **significant drop in accuracy** when validating on the target device compared to the desktop validation. The next steps involve investigating this discrepancy and improving deployment performance.

### Details

The **CY8CKIT-062-BLE** was chosen as the development kit.

A new project was created using the **Machine Learning Neural Network Profiler** template in ModusToolbox.

The default example code was used as a baseline for validation and deployment.



### ML Model Import & Configuration:

The **ML Configurator** was used to import and validate the model.

Model variants were generated in .c and .h formats for deployment.

Two inference engine options were available:

- o **IFX** (with advanced scratch memory optimization).
- o **TFLM** (TensorFlow Lite Micro, with quantization options).

The generated models were stored in the mtb\_ml\_models folder.

### Pre-Validation & Testing:

The model was validated using different datasets, including **randomly generated data, NPZ files, folder-based datasets, and CSV files.**

The **Mean Absolute Error (MAE)** was analyzed for validation.

Validation results were filtered based on model performance, focusing on **accuracy and memory usage.**

**Pass condition:** Accuracy | 98% and scratch memory usage < 1MB.

### Validation on Desktop vs Target:

**Desktop Validation:** 99% accuracy (99/100 correct predictions).

**Target Validation (MCU):** Only 16/100 correct predictions, showing a major performance drop.

### Deployment & Code Integration

Deployment involved modifying **main.c** and **Makefile** in Eclipse IDE.

The **Makefile** automates the process of selecting and deploying the model defined in the ML Configurator.

The **main.c** file handles clock setup, power configuration, and model loading.

## Conclusion

This project provided valuable insights into deploying machine learning models on embedded systems using ModusToolbox. I learned how to configure and validate models using the ML Configurator, optimize them for inference on an MCU, and troubleshoot deployment challenges. A key takeaway was understanding the impact of quantization and memory constraints on model accuracy, as seen in the significant drop when running on the target device. Additionally, engaging with the Infineon community reinforced the importance of seeking expert guidance for debugging hardware-related ML issues.

The project successfully set up and validated an ML model on the **CY8CPROTO-063-BLE** but faced **significant accuracy degradation** on the MCU. The next steps involve diagnosing the root cause by:

1. Comparing raw inputs fed into the model on desktop vs. target device to check for discrepancies.
2. Investigating inference issues or quantization-related problems.
3. I have posted a request for help in the Infineon community to understand why this problem is occurring and am waiting for their response.

## Planned Work

### Summary

This literature survey on exploring security vulnerabilities in TinyML models, particularly focusing on model extraction, evasion attacks, electromagnetic side-channel attacks, and hardware security mechanisms. The studies investigated various embedded platforms such as ESP32, Raspberry Pi and STM32 to assess how TinyML models can be compromised and what techniques exist to mitigate these risks. Key findings include the susceptibility of models to adversarial attacks, model theft via electromagnetic emissions, and the effectiveness of trusted execution environments (TEE) and attestation mechanisms in enhancing security.

### Details

TinyML models are highly vulnerable to various security threats, including model extraction, evasion attacks, and side-channel attacks. Adversarial attacks, such as FGSM,

can significantly degrade model performance, causing misclassification rates as high as 95.76%. Model extraction attacks allow adversaries to replicate models with high accuracy using only a fraction of the training data. Electromagnetic side-channel attacks can even steal model parameters by analyzing emissions during inference. To address these risks, Trusted Execution Environments (TEE) and IP protection mechanisms have been proposed, utilizing memory isolation and secure attestation to safeguard models. Optimization techniques, such as flash and parallel processing, help reduce latency while maintaining security, ensuring TinyML models remain both efficient and protected.

## Conclusion

The survey highlights that TinyML models, despite their constrained environment, remain susceptible to multiple security threats, including adversarial attacks, side-channel leaks, and hardware-level vulnerabilities. While techniques like TEE, attestation mechanisms, and PMP offer promising security enhancements, their implementation often comes with trade-offs in memory usage and runtime overhead. Future research should focus on **lightweight yet effective security solutions** that balance protection and performance, ensuring TinyML models can be deployed securely in real-world applications.

# END